

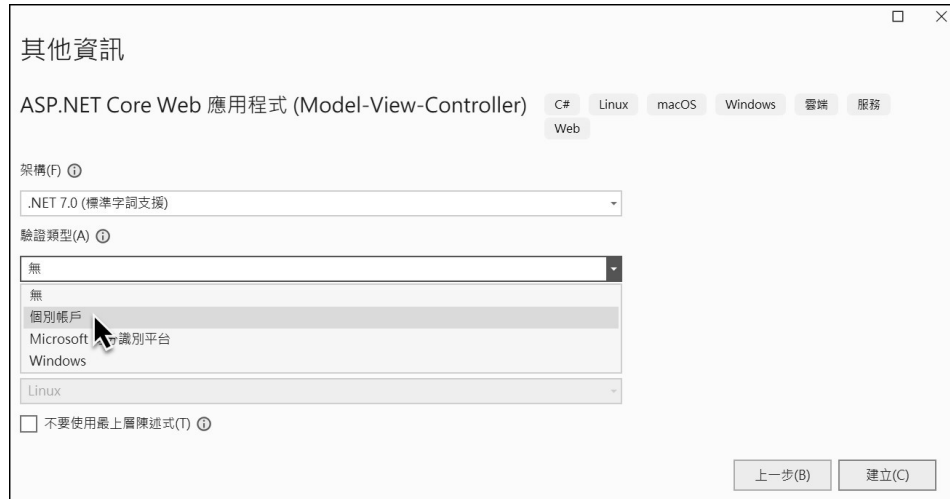


圖 2-23 個別帳戶驗證類型

以上可視 MVC 網站需求而選擇不同身分驗證類型，但因本書各章專案不需管制網站資源存取，維持預設的「無」驗證模式即可。

範例 2-3 使用 ASP.NET Identity 建立使用者帳號及存取管制

以下透過 ASP.NET Identity 提供帳號註冊、登入及驗證等功能，並結合 Authorize 屬性來管制使用者對 Action 的存取權限，請參考 Mvc7_Identity 專案。

step01 建立「Mvc7_Identity」MVC 專案，驗證類型選擇「個別帳戶」，便會建立 ASP.NET Identity 的相關程式

step02 修改 appsettings.json 的 DefaultConnection 資料庫連線設定，將資料庫名稱改為「IdentityDB」

```
{
  ...,
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=IdentityDB;"
  }
}
```

step 03 以 Migration 更新資料庫

由於 ASP.NET Core Identity 有一 Migration 未更新至資料庫，選擇以下一種方式更新：

1. 在 Visual Studio 的 NuGet 套件管理器主控台中執行「Update-Database」
2. 在命令視窗執行「dotnet ef database update」

用資料庫管理工具檢視 localdb，可看見「IdentityDB」資料庫。

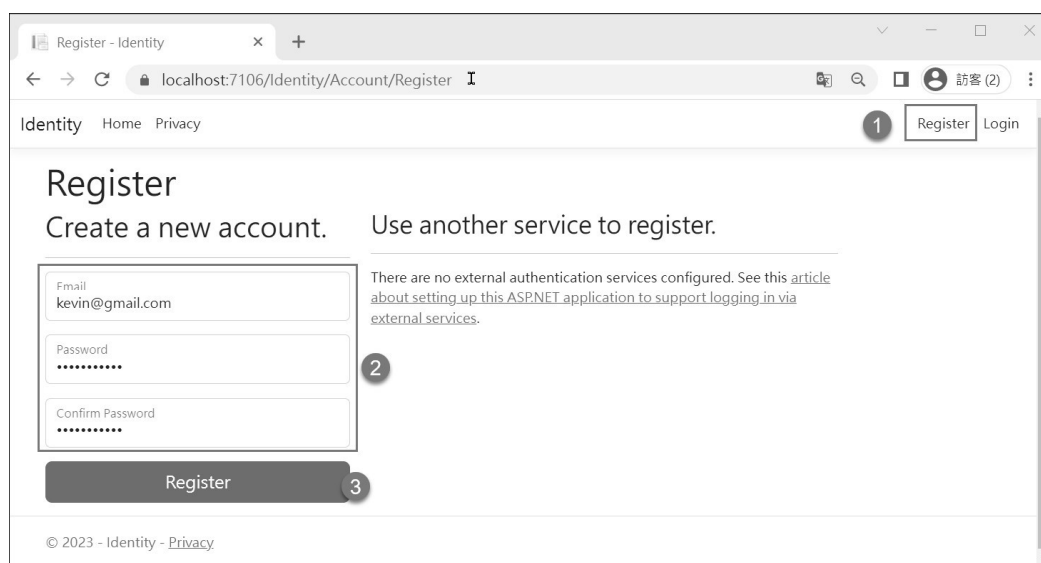
step 04 以 F5 執行，點擊網頁右上方的註冊，註冊 kevin@gmail.com、mary@gmail.com 和 john@gmail.com 三個使用者帳號

圖 2-24 註冊使用者帳號

註冊成功後的帳號無法立即登入，需要真實的 E-Mail 啟用認證才能登入，但權宜之計，是將 IdentityDB 的 AspNetUser 資料表中，三位使用者的 EmailConfirmed 欄位由 False 改為 True，偽裝已通過電子郵件驗證，即可登入。

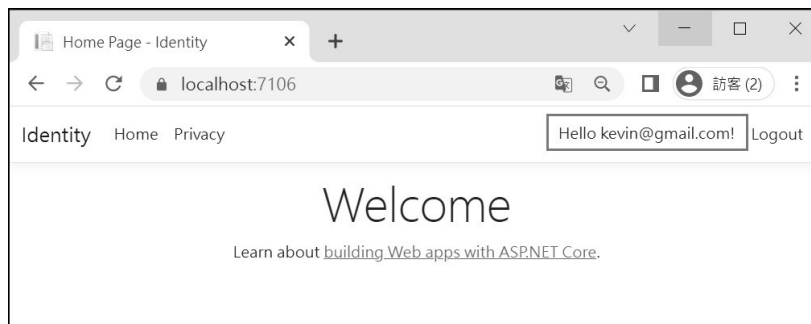


圖 2-25 帳號登入成功

step05 將 Program.cs 中需要驗證帳號設定移除

Identity 預設建立的帳號皆需要做 Email 信件驗證，但這對於本機練習或測試麻煩了些，可在 Program.cs 中將 `RequireConfirmedAccount` 改為 `false`：

```
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
    options.SignIn.RequireConfirmedAccount = false)
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

step06 設定 Home 控制器及 Actions 的存取權限

在控制器及 Actions 套用 `[Authorize]` 屬性授予存取權限：

 Controllers/HomeController.cs

```
...
using Microsoft.AspNetCore.Authorization;
[Authorize] ← 在控制器層級設定授權
public class HomeController : Controller
{
    ...
    [AllowAnonymous] ← 允許匿名存取
    public IActionResult Index()
    {
        return View();
    }

    public IActionResult Privacy()
```

```
{
    // 利用 Identity.Name 判斷是否為特定使用者
    if (User.Identity.Name != "Kevin@gmail.com")
    {
        return Content($"{User.Identity.Name}無權存取此 Action 動作方法!");
    }
    return View();
}

// 授權予特定角色
[Authorize(Roles = "Admin, Supervisor")]
public IActionResult Contact()
{
    return View();
}
}
```

說明：

1. 以上在 Controller 及 Index 動作方法套用 [Authorize] 及 [AllowAnonymous] 屬性，前者限制只有通過驗證或授權的使用者能存取，後者則無條件開放匿名存取
2. [Authorize] 作用是使用者登入後就可以存取
3. 在 Privacy() 以 User.Identity.Name 判斷是否為特定使用者，才允許存取
4. [Authorize(Roles = "Admin, Supervisor")] 限制使用者必須屬於指定的角色才能存取

按 F5 執行，分別以未登入的匿名使用者、kevin、mary 及 john 瀏覽 Index、Privacy 和 Contact，以檢驗是否受到不同的存取限制。

step 07 檢視 ASP.NET Identity 身分驗證資料庫

在 Visual Studio 的【檢視】→【SQL Server 物件總管】→ (localdb)\MSSQLLocalDB → 找尋「IdentityDB」資料庫 → 檢視「dbo.AspNetUsers」資料表，使用者註冊的帳號資料就在其中。



圖 2-26 檢視使用者帳號資料

2-8 用 LibMan 管理前端函式庫

一上代 ASP.NET MVC 5 用 Visual Studio 管理專案，安裝 jQuery 或 Bootstrap 函式庫是透過 NuGet 套件管理員，但在 ASP.NET Core 世代是用 LibMan 負責前端函式庫的安裝、更新或移除管理。而用前端函式庫是指 JavaScript 或 CSS Library，例如 jQuery、Chart.js 或 Bootstrap 函式庫，LibMan 會從 CDN 下載，有 cdnjs、jsdelivr 和 unpkg 三個 CDN 來源，以及一個 filesystem，CDN 預設會使用 cdnjs。

LibMan 使用上又分兩種，一是經由 Visual Studio 的 GUI 介面，另一是透過 .NET CLI 命令，在此介紹前者，後者在第三章會說明。例如用 Visual Studio 建立 ASP.NET Core 7 MVC 專案，在 wwwroot/lib 有 bootstrap 和 jquery 兩個子目錄，它們版本分別是 5.2.0 和 3.6.1，以下說明如何用 LibMan 來升級到最新版本 5.2.3 和 3.6.3。

範例 2-4 用 LibMan 升級與管理 Bootstrap 和 jQuery 前端函式庫

以下用 LibMan 升級與管理 Bootstrap 和 jQuery 前端函式庫。

step01 在專案按滑鼠右鍵→【加入】→【用戶端程式庫】，會出現新增用戶端程式庫